

Посібник з навчання

Часова кореляція кіберзагроз — зберіть, навчіть і отримайте готову до розгортання мережу

Повний покроковий маршрут для копіювання навчальної половини конвеєра: встановіть інструментарій, зберіть платформу, запустіть демо, навчіть модель на еталонних і реальних даних і отримайте готову до розгортання мережу, яку споживає Посібник з розгортання.

Документ	Посібник з навчання моделі
Продукт	Модуль кібербезпеки Jneopallium (Демо 06)
Модель	cybersecurity-temporal-threat-correlator 1.0.0
Тренер	scripts/demo-cybersecurity-training/ train_temporal_model.py
Вихід	Готова до розгортання конфігурація JNeopallium
Автор	Дмитро Раковський — Харків, Україна
Дата	23 червня 2026
Ліцензія	BSD 3-Clause

Зміст

1. Чого ви досягнете
2. Де закінчується навчання і починається розгортання
3. Передумови та системні вимоги
4. Крок 1 — Отримати код і зібрати
5. Крок 2 — Запустити демо кібербезпеки
6. Крок 3 — Зрозуміти згенеровані результати демо
7. Крок 4 — Навчити еталонну модель
8. Крок 5 — Навчання у промисловому масштабі
9. Крок 6 — Навчання на зовнішніх даних (режим маніфесту)
10. Крок 7 — Навчання на сирих LANL + ToN_IoT
11. Крок 8 — Згенеровані артефакти: готова до розгортання мережа
12. Крок 9 — Перевірити запуск за контрольними критеріями
13. Усунення несправностей
14. Додаток — шпаргалка навчання

1 Чого ви досягнете

До кінця цього посібника ви матимете: зібрану платформу Jнеорallium; робоче демо кібербезпеки, що видає рекомендаційний JSONL; свіжонавчену модель часової кореляції загроз; і — ключовий результат — **повну, готову до розгортання мережу JНеорallium** (файли конфігурації рівнів і нейронів зі справжніми класами часу виконання, вбудованими навченими вагами та фіксованим запобіжником), яку напряду завантажує супровідний **Посібник з розгортання**.

Стеля безпеки для всього конвеєра

Усе, що робить навчена модель, працює в режимі ADVISORY. Вона може рекомендувати кандидатів на розслідування чи карантин, але не повинна ізолювати вузли чи блокувати трафік. Тренер кодує це прямо: рівень відповіді позначено ADVISORY, а жорсткий запобіжник — це фіксована конфігурація, ніколи не навчена.

2 Де закінчується навчання і починається розгортання

Конвеєр має дві чисті половини, і цей посібник охоплює першу:

Фаза	Що відбувається	Документ
Навчання (цей посібник)	Зборка, запуск демо, навчання на даних, видача мережі	Посібник з навчання
Розгортання	Завантаження виданої мережі, налаштування джерел подій, запуск worker	Посібник з розгортання

Передача між двома половинами — це єдина тека згенерованих артефактів (Крок 8). Навчання її **створює**; розгортання її **споживає**. Між ними нічого не пишеться вручну — тренер записує готову для JНеорallium конфігурацію рівнів і нейронів, тож ті самі файли, що фіксують ваш запуск навчання, є файлами, з яких завантажується worker.

3 Передумови та системні вимоги

Інструментарій

Інструмент	Версія	Навіщо
Java JDK	17 або новіша (LTS)	Платформа орієнтована на базовий API Java 17
Apache Maven	3.9 або новіша	Багатомодульна збірка master + worker
Python	3.10+ (тестовано 3.13)	Запускає тренер моделі; сторонні пакети не потрібні
Git	будь-яка свіжа	Клонування репозиторію

Апаратне забезпечення

- **Демо / пілот:** будь-який сучасний ноутбук. Еталонний тренер зручно працює в одному JVM, а набір даних настільного масштабу займає кілька десятків мегабайтів.
- **Еталонний запуск промислового масштабу:** тренер фіксує *логічну* ціль корпусу 100 ГіБ без запису 100 ГіБ на диск, тож усе одно працює на робочій станції. Реальне навчання на зовнішніх даних масштабується відповідно до обсягу, який ви подаєте.

Перевірте інструментарій

```
java -version      # очікуємо 17 або новіше
mvn -version       # очікуємо 3.9 або новіше
python --version   # очікуємо 3.10 або новіше
git --version
```

4 Крок 1 — Отримати код і зібрати

Клонуйте репозиторій і зберіть обидва модулі. Збірка встановлює артефакти master і worker у ваш локальний репозиторій Maven.

```
git clone https://github.com/rakovpublic/jneopallium.git
cd jneopallium
mvn clean install
```

Успішна збірка компілює платформу, виконує набір тестів (зокрема `SecurityModuleTest`) і встановлює `com.rakovpublic.jneopallium:worker:1.0`. Worker уже містить справжні класи нейронів, процесорів і сигналів безпеки, на які посилатиметься навчена мережа, тож для навчання не потрібні додаткові написані вручну класи.

5 Крок 2 — Запустити демо кібербезпеки

Набір повних демо запускається через справжній вхідний шлях worker. Запустіть увесь набір (Демо 06 — це сценарій кібербезпеки):

Windows (PowerShell)

```
powershell -ExecutionPolicy Bypass -File scripts/demo-fullrun/run_all_fullrun_demos.ps1
```

Linux / macOS (bash)

```
scripts/demo-fullrun/run_all_fullrun_demos.sh
```

Сценарій кібербезпеки корелює контекст автентифікації, процесів, DNS, мережевих потоків, активів, кіберрозвідки та обслуговування для трьох сутностей — впорядкований ланцюг атаки, доброякісний шаблон обслуговування та шаблон повільного витоку — і записує рекомендаційні результати без жодної блокувальної дії.

6 Крок 3 — Зрозуміти згенеровані результати демо

Типові докази повного запуску записуються в target/:

```
target/jneopallium-fullrun-demos/summary.json
target/jneopallium-fullrun-demos/demo-06-cybersecurity-kafka-triage/
```

Вихід Демо 06 — це рекомендаційний JSONL. Кожен рядок несе апостеріорну ймовірність, вплив, впевненість послідовності, запобіжник кіберрозвідки, запобіжник обслуговування та стан заморожування базової лінії. Очікувана детермінована поведінка така:

Потік	Очікуваний вердикт
Впорядкований ланцюг атаки	TEMPORAL_THREAT_ADVISORY (базова лінія заморожена під час атаки)
Активність у вікні обслуговування	CONTEXT_SUPPRESSED_OBSERVATION (нижча оцінка, але в аудиті)
Повільний вихідний трафік	LOW_AND_SLOW_CORRELATION
Будь-який потік	Жодного активного блокувального результату не видається

7 Крок 4 — Навчити еталонну модель

Тренер не потребує сторонніх пакетів Python. Найпростіший запуск використовує вбудований детермінований багатоджерельний еталонний корпус:

```
python scripts/demo-cybersecurity-training/train_temporal_model.py
```

Він видає повний набір артефактів у ресурси worker (повний перелік і призначення кожного файлу — у Кроці 8). Після завершення тренер друкує зведення JSON: вибраний поріг рішення, тестовий F1 на відкладений вибірці й оцінений розмір корпусу. Запуск завершується помилкою (ненульовий код), якщо тестовий F1 нижчий за поріг `--min-test-f1` (за замовчуванням 0.85), тож зламаний конвеєр не може тихо відвантажити слабку модель.

8 Крок 5 — Навчання у промисловому масштабі

Цей запуск фіксує *логічну* ціль корпусу 100 ГіБ, детерміновано реплікуючи еталонні кампанії, водночас тримаючи припасовану вибірку обмеженою, тож усе одно завершується на робочій станції. Він вправляє метадані масштабування та запобіжники.

Windows (PowerShell)

```
python scripts\demo-cybersecurity-training\train_temporal_model.py `
  --reference-multiplier 1000 `
  --target-corpus-bytes 100gb `
  --max-corpus-bytes 100gb `
  --max-train-windows-per-epoch 4096
```

Linux / macOS (bash)

```
python scripts/demo-cybersecurity-training/train_temporal_model.py \
--reference-multiplier 1000 \
--target-corpus-bytes 100gb \
--max-corpus-bytes 100gb \
--max-train-windows-per-epoch 4096
```

Що означають прапорці

--reference-multiplier детерміновано розширює припасовану вибірку. --target-corpus-bytes — це логічна ціль промислового масштабу, записана в артефакти. --max-corpus-bytes — жорсткий запобіжник: запуск переривається, замість роздувати репозиторій. --max-train-windows-per-epoch обмежує роботу за епоху для дуже великих розширених корпусів.

9 Крок 6 — Навчання на зовнішніх даних (режим маніфесту)

Для реальної якості потрібно навчати на зовнішніх канонічних даних. Режим маніфесту зберігає ту саму політику розбиття без витоку, читаючи ваші власні джерела JSONL або CSV.

```
python scripts/demo-cybersecurity-training/train_temporal_model.py \
--manifest scripts/demo-cybersecurity-training/dataset-manifest-template.json \
--output-dir worker/src/main/resources/model/cybersecurity-temporal \
--max-corpus-bytes 100gb
```

Канонічний контракт події

Кожен рядок навчання має бути канонічним JSONL або CSV з такими полями:

```
dataset, entity_id, event_tick, source, event_type, technique,
evidence_confidence, threat_intel_confidence, asset_criticality,
maintenance_active, malicious, campaign_id, host_group, attack_type, split
```

Правило	Чому це важливо
event_tick — час події, а не час прийому	Дає рушію відтворити справжню послідовність атаки
campaign_id групує один інцидент чи зіставлений доброякісний період	Тримає пов'язані події разом для розбиття
split — за часом / кампанією / вузлом / типом атаки	Ніколи не рандомізуйте рядки — це просочує дублікати й завищує оцінки
maintenance_active — м'яка контекстна ознака	Знижує терміновість, але ніколи не видаляє докази
malicious — істинна мітка лише для навчання	Висновок під час роботи ніколи не повинен від неї залежати

10 Крок 7 — Навчання на сирих LANL + ToN_IoT

Окремий шаблон приймає сирі публічні набори даних безпосередньо. Завантажте файли й покладіть їх у зовнішні теки:

```
scripts/demo-cybersecurity-training/external/lanl/
  auth.txt.gz proc.txt.gz dns.txt.gz flows.txt.gz redteam.txt.gz

scripts/demo-cybersecurity-training/external/toniot/
  ... виводити теки CSV ToN_IoT ...
```

- LANL Comprehensive Multi-Source Cyber-Security Events: <https://csr.lanl.gov/data/cyber1/>
- Набори даних ToN_IoT: <https://research.unsw.edu.au/projects/toniot-datasets>

Потім запустить шаблон LANL + ToN_IoT (поточкова обробка стиснених файлів замість завантаження в пам'ять):

```
python scripts\demo-cybersecurity-training\train_temporal_model.py `
  --manifest scripts\demo-cybersecurity-training\dataset-manifest-lanl-toniot-
  template.json `
  --output-dir worker\src\main\resources\model\cybersecurity-temporal `
  --max-corpus-bytes 100gb `
  --max-train-windows-per-epoch 4096
```

Шаблон також приймає сирі файли LANL (lanl-auth, lanl-proc, lanl-dns, lanl-flow, lanl-redteam) і поширені файли CSV ToN_IoT (toniot-network, toniot-windows, toniot-linux). Налаштуйте maxRows, startTick, endTick і sampleModulo у маніфесті перед навчанням на великих вистах. Тримайте ліцензії наборів даних та інструкції зі завантаження поза згенерованими артефактами.

11 Крок 8 — Згенеровані артефакти: готова до розгортання мережа

Це передача конвеєра. Кожен запуск навчання записує теку конфігурації у стилі JNeorallium, готову до розгортання як є, — не просто абстрактну модель, а власне мережу рівнів і нейронів, з якої завантажуються worker. Створюється десять файлів:

Артефакт	Що це
trained-temporal-threat-model.json	Компактна навчена модель: ознаки, скейлер, ваги, зсув, поріг, метрики
model-descriptor.json	Опис усієї мережі: 5 рівнів, 8 нейронів, мапа частот, класи джерел
layer-0.json	Межа багатоджерельного входу подій безпеки (канонічні входи на родину джерел)
layer-1-fast-evidence.json	Швидкі рецептори доказів — 4 нейрони (потік, сигнатура, процес, базова лінія)
layer-2-temporal-correlation.json	Навчений часовий корелятор — вбудовує ваги, гейти, дендрити
layer-3-response-planning.json	Планування рекомендаційної відповіді + фіксований жорсткий запобіжник
result-layer.json	Рівень виводу рекомендацій безпеки
trained-model-update.json	Останній знімок навчання: статус, контрольна сума, ваги, зведення
quantitative-summary.json	Масштаб, метрики та найвагоміші позитивні /

	негативні ваги ознак
source-mapping.json	На які типізовані сигнали відображається кожне джерело даних

Мережа, яку будує тренер

model-descriptor.json фіксує п'ятирівневу мережу з восьми справжніх нейронів, з 34 навчуваними вагами та одним зсувом — усі посилаються на конкретні класи часу виконання з пакета безпеки worker:

Рівень	Роль	Справжні класи нейронів
0 — Вхід	Межа багатоджерельних подій	(без об'єктів нейронів; лише канонічні входи)
1 — Швидкі докази	Дешева миттєва оцінка	NetworkFlowNeuron, SignaturePatternNeuron, ProcessBehaviourNeuron, EntityBehaviourBaselineNeuron
2 — Кореляція	Навчена часова кореляція	TemporalThreatCorrelationNeuron
3 — Планування + запобіжник	Смуги рекомендацій + запобіжник	ResponsePlanningNeuron, ResponseGateNeuron
4 — Результат	Рекомендаційний вивід	ResponsePlanningNeuron (результат)

Рівень кореляції (layer-2-temporal-correlation.json) вбудовує навчені логістичні ваги як іменовані **ваги дендритів**, скейлер стандартизації, поріг рішення (0.2), п'ять часових гейтів послідовності та блок baselineAdaptation, що заморожує навчання, коли апостеріорна сягає **0.3** або впевненість сигнатури сягає **0.8**, і адаптується лише з довірених доброякісних періодів. Опис також містить signalFrequencyMap, що дає кожному сигналу його каданс циклу — швидкі рецептори щоцикл, гіпотези й запити карантину кожен другий цикл, звіти про інциденти кожен п'ятий — дизайн типізованих часових масштабів, втілений конкретно.

Чому це важливо

Оскільки навчання видає справжню, придатну до перевірки, готову до розгортання конфігурацію, між «тим, що ми навчили» і «тим, що ми запускаємо», немає кроку перекладу. Посібник з розгортання просто спрямовує worker на цю теку. Кожна вага, гейт і поріг, які ви бачите у звіті, — це саме те, що виконується в промислі.

12 Крок 9 — Перевірити запуск за контрольними критеріями

Перш ніж вважати будь-який запуск придатним до розгортання, переконайтеся, що промислові критерії пройдено:

1. Усі очікувані родини джерел присутні в source-mapping.json.
2. Тестовий F1 не нижчий за поріг --min-test-f1.

3. Рівень хибних спрацювань у межах бюджету на вузол і на орендаря.
4. Кошки калібрування показують монотонне впорядкування ризику (вища оцінка → вищий реальний рівень позитивів).
5. Середній час до виявлення нижчий за вашу ціль реагування на інциденти.
6. П'ять файлів рівнів і `trained-model-update.json` записано, і вони посилаються на очікувані класи часу виконання.
7. Згенеровані артефакти містять контрольну суму, кількості джерел, політику розбиття, метрики та поріг.

Перевірка відтворюваності

Еталонний запуск детермінований. Повторний запуск тієї самої команди має дати ту саму контрольну суму навчання (`trained-model-update.json`) і ті самі метрики. Розбіжність у будь-чому з них — сигнал, що змінилися вхідні дані чи інструментарій.

13 Усунення несправностей

Симптом	Імовірна причина й виправлення
<code>java -version</code> показує <code>< 17</code>	Встановіть JDK 17+ і вкажіть на нього <code>JAVA_HOME</code>
Збірка падає на JDK 17 з попередженнями рефлексії	Дотримайте мінімумів плагінів Maven з README; вживайте <code><release>17</release></code>
Тренер виходить з помилкою «test F1 below required»	Розбиття чи дані заслабкі; перевірте політику розбиття маніфесту та <code>--min-test-f1</code>
«estimated canonical corpus exceeds --max-corpus-bytes»	Знизьте <code>--reference-multiplier/--target-corpus-bytes</code> або свідомо підніміть запобіжник
Джерело відсутнє в <code>source-mapping.json</code>	Перевірте шаблони маніфесту та <code>requireAllSources</code> ; підтвердьте, що файли завантажено
Файли рівнів не записано	Переконайтеся, що <code>--output-dir</code> доступний для запису; тренер пише туди всі десять артефактів
Проблеми з кирилицею / кодуванням у CSV	Зберігайте джерела як UTF-8; читач очікує UTF-8 JSONL/CSV

14 Додаток — шпаргалка навчання

Збірка

```
git clone https://github.com/rakovpublic/jneopallium.git && cd jneopallium && mvn clean install
```

Запуск набору демо

```
# Windows
powershell -ExecutionPolicy Bypass -File scripts/demo-fullrun/run_all_fullrun_demos.ps1
# Linux / macOS
```

```
scripts/demo-fullrun/run_all_fullrun_demos.sh
```

Навчання (еталон / масштаб / зовнішнє / сире)

```
# еталон
python scripts/demo-cybersecurity-training/train_temporal_model.py

# еталон промислового масштабу
python scripts/demo-cybersecurity-training/train_temporal_model.py \
  --reference-multiplier 1000 --target-corpus-bytes 100gb \
  --max-corpus-bytes 100gb --max-train-windows-per-epoch 4096

# зовнішній маніфест
python scripts/demo-cybersecurity-training/train_temporal_model.py \
  --manifest scripts/demo-cybersecurity-training/dataset-manifest-template.json

# сирі LANL + ToN_IoT
python scripts/demo-cybersecurity-training/train_temporal_model.py \
  --manifest scripts/demo-cybersecurity-training/dataset-manifest-lanl-toniot-
template.json \
  --max-corpus-bytes 100gb --max-train-windows-per-epoch 4096
```

Модуль кібербезпеки Jнеорallium · Демо 06 · Посібник з навчання. Тренер видає готову до розгортання мережу JНеорallium; як її запустити — у Посібнику з розгортання. Режим безпеки: ADVISORY. Ліцензія: BSD 3-Clause.